

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 –Dataset Intégration Task

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO3	Assessment:	Tool 3 - Dataset Intégration Task
Weightage:		Total Marks:	
CO3 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex. (BL5 and BL6).			
Student Name:	BHARANI DHARNA S	Reg. No.:	192424141
Section / Batch:	D / 2024	Date of Submission:	13/08/2026
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I BHARANI DHARAN S(192424141)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: BHARANNI DHARAN S

Dataset Intégration Task

Question 2 – Employee Dataset Integration

Aim

To develop a Python program to integrate employee data from **HR, Payroll, and Attendance datasets** by performing data formatting, RegEx validation, duplicate detection, fuzzy matching, normalization, outlier detection, and missing-value handling.

Methodology

The following operations are performed:

1. Read HR, Payroll, and Attendance CSV files.
2. Standardize column names.
3. Clean and normalize employee names.
4. Validate email IDs using **RegEx**.
5. Detect exact duplicate records.
6. Use **fuzzy matching** to identify similar employee names.
7. Standardize department values.
8. Convert different date formats into a common format.
9. Detect salary outliers using the **IQR method**.
10. Handle missing department and other values.
11. Merge the three datasets using Employee ID.
12. Save the final cleaned dataset.
13. Test the script using new employee records.

Python Program

```
import pandas as pd
import re
from difflib import SequenceMatcher

# -----
# 1. Read datasets
# -----

hr = pd.read_csv("hr.csv")
payroll = pd.read_csv("payroll.csv")
attendance = pd.read_csv("attendance.csv")

# -----
# 2. Standardize column names
# -----

def standardize_columns(df):
    df.columns = (
        df.columns
        .str.strip()
        .str.lower()
        .str.replace(" ", "_")
    )
    return df

hr = standardize_columns(hr)
```

```

payroll = standardize_columns(payroll)
attendance = standardize_columns(attendance)

# Rename columns into common names

hr.rename(columns={
    "emp_id": "employee_id",
    "employee_name": "name",
    "email_id": "email",
    "dept": "department"
}, inplace=True)

payroll.rename(columns={
    "empid": "employee_id",
    "emp_name": "name",
    "salary_amount": "salary"
}, inplace=True)

attendance.rename(columns={
    "empid": "employee_id",
    "employee": "name",
    "attendance_date": "date"
}, inplace=True)

# -----
# 3. Clean employee names
# -----

def clean_name(name):
    if pd.isna(name):
        return None

    name = str(name).strip().lower()
    name = re.sub(r"^[^a-zA-Z\s]", "", name)
    name = re.sub(r"\s+", " ", name)

    return name.title()

for df in [hr, payroll, attendance]:
    if "name" in df.columns:
        df["name"] = df["name"].apply(clean_name)

# -----
# 4. Validate email using RegEx
# -----

email_pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"

def validate_email(email):
    if pd.isna(email):

```

```

        return None

    email = str(email).strip().lower()

    if re.match(email_pattern, email):
        return email
    return None

if "email" in hr.columns:
    hr["email"] = hr["email"].apply(validate_email)

# -----
# 5. Remove duplicate employees
# -----

hr = hr.drop_duplicates(subset=["employee_id"])
payroll = payroll.drop_duplicates(subset=["employee_id"])
attendance = attendance.drop_duplicates(
    subset=["employee_id", "date"]
)

# -----
# 6. Standardize departments
# -----

department_map = {
    "hr": "Human Resources",
    "human resource": "Human Resources",
    "human resources": "Human Resources",
    "it": "Information Technology",
    "information technology": "Information Technology",
    "finance": "Finance",
    "fin": "Finance",
    "sales": "Sales",
    "marketing": "Marketing"
}

if "department" in hr.columns:
    hr["department"] = (
        hr["department"]
        .astype("string")
        .str.strip()
        .str.lower()
        .map(department_map)
    )

    hr["department"] = hr["department"].fillna("Unknown")

# -----
# 7. Standardize dates

```

```

# -----

if "date" in attendance.columns:
    attendance["date"] = pd.to_datetime(
        attendance["date"],
        errors="coerce",
        dayfirst=True
    )

    attendance["date"] = attendance["date"].dt.strftime(
        "%Y-%m-%d"
    )

# -----
# 8. Normalize salary
# -----

if "salary" in payroll.columns:
    payroll["salary"] = (
        payroll["salary"]
        .astype(str)
        .str.replace(",", "", regex=False)
        .str.replace("₹", "", regex=False)
        .str.replace("$", "", regex=False)
    )

    payroll["salary"] = pd.to_numeric(
        payroll["salary"],
        errors="coerce"
    )

# -----
# 9. Detect salary outliers
#   using IQR
# -----

Q1 = payroll["salary"].quantile(0.25)
Q3 = payroll["salary"].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

payroll["salary_outlier"] = (
    (payroll["salary"] < lower_limit) |
    (payroll["salary"] > upper_limit)
)

# Replace outlier salary with median

```

```

median_salary = payroll["salary"].median()

payroll.loc[
    payroll["salary_outlier"],
    "salary"
] = median_salary

# -----
# 10. Fuzzy matching
# -----

def similarity(name1, name2):
    return SequenceMatcher(
        None,
        str(name1).lower(),
        str(name2).lower()
    ).ratio()

# Compare HR names with Payroll names

for _, hr_row in hr.iterrows():

    best_match = None
    best_score = 0

    for _, pay_row in payroll.iterrows():

        score = similarity(
            hr_row["name"],
            pay_row["name"]
        )

        if score > best_score:
            best_score = score
            best_match = pay_row["name"]

    if best_score >= 0.85:
        print(
            f"Fuzzy match: {hr_row['name']} -> "
            f"{best_match} ({best_score:.2f})"
        )

# -----
# 11. Integrate datasets
# -----

employee_master = pd.merge(
    hr,
    payroll[["employee_id", "salary"]],

```

```

        on="employee_id",
        how="outer"
    )

    employee_master = pd.merge(
        employee_master,
        attendance.groupby("employee_id")
            .size()
            .reset_index(name="attendance_records"),
        on="employee_id",
        how="left"
    )

# -----
# 12. Missing-value handling
# -----

employee_master["department"] = (
    employee_master["department"]
    .fillna("Unknown")
)

employee_master["attendance_records"] = (
    employee_master["attendance_records"]
    .fillna(0)
)

employee_master["salary"] = (
    employee_master["salary"]
    .fillna(employee_master["salary"].median())
)

employee_master["name"] = (
    employee_master["name"]
    .fillna("Unknown")
)

employee_master["email"] = (
    employee_master["email"]
    .fillna("Not Available")
)

# -----
# 13. Save cleaned dataset
# -----

employee_master.to_csv(
    "cleaned_employee_master.csv",
    index=False
)

```

```
print("\nEmployee integration completed successfully!")
print(employee_master)
print("\nSaved as cleaned_employee_master.csv")
```

Example Input

HR Dataset

emp_id	employee_name	email_id	dept
E101	Ravi Kumar	ravi@gmail.com	IT
E102	Priya Devi	priya@gmail.com	HR
E103	Arun Raj	invalid-email	Finance

Payroll Dataset

empid	emp_name	salary_amount
-------	----------	---------------

E101	Ravi Kumar	₹45000
E102	Priya Davi	50000
E103	Arun Raj	9999999

Attendance Dataset

empid	employee	attendance_date
-------	----------	-----------------

E101	Ravi Kumar	01/08/2026
E102	Priya Devi	2026-08-02
E103	Arun Raj	03-08-2026

Expected Result

The final dataset will contain standardized information such as:

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
PS C:\Users\sambh\Videos\employee_master.csv>	py_employee_integration.py			
0	Anitha C	anitha.c@gmail.com	Sales	2024/10/05
1	Arun Kumar	arun.kumar@gmail.com	IT	12-01-2024
2	Divya T	divya.t@gmail.com	Marketing	2024/06/10
3	Karthik P	karthik.p@gmail.com	Unknown	05/05/2024
4	Lakshmi V	lakshmi.v@gmail.com	HR	2024-08-12
5	Meena R	meena.r@gmail.com	Sales	2024-04-20
6	Priya S	priya.s@gmail.com	HR	2024/02/15
7	Rahul M	rahul.m@gmail.com	Finance	15-03-2024
8	Ravi K	ravi.k@gmail.com	IT	18-07-2024
9	Suresh B	suresh.b@gmail.com	Finance	01/09/2024
10	Vijay R	vijay.r@gmail.com	Unknown	15-11-2024

The invalid email is converted to **Not Available**, department names are standardized, salary outliers are detected, and similar names such as **Priya Devi / Priya Davi** can be identified using fuzzy matching.

Question 3 – Product Dataset Integration

Aim

To develop a Python-based data cleaning and integration system for **Supplier, Warehouse, and Online Store datasets** using RegEx validation, fuzzy matching, duplicate detection, category standardization, price outlier detection, and numerical normalization.

Methodology

1. Read all three product datasets.
2. Standardize column names.
3. Clean product names.
4. Validate product codes using **RegEx**.
5. Detect duplicate products.
6. Use fuzzy matching for spelling errors.
7. Standardize product categories.
8. Convert prices and stock quantities into numeric values.

9. Detect price outliers using IQR.
10. Normalize numerical values.
11. Integrate all datasets.
12. Generate **CSV and JSON** master files.

Python Program

```
import pandas as pd
import re
from difflib import SequenceMatcher

# -----
# 1. Read datasets
# -----

supplier = pd.read_csv("supplier.csv")
warehouse = pd.read_csv("warehouse.csv")
online = pd.read_csv("online_store.csv")

# -----
# 2. Standardize column names
# -----

def clean_columns(df):

    df.columns = (
        df.columns
        .str.strip()
        .str.lower()
        .str.replace(" ", "_")
    )

    return df

supplier = clean_columns(supplier)
warehouse = clean_columns(warehouse)
online = clean_columns(online)

# Rename columns

supplier.rename(columns={
    "product_code": "code",
    "product_name": "name",
    "unit_price": "price",
    "category_name": "category"
}, inplace=True)

warehouse.rename(columns={
    "prod_code": "code",
    "prod_name": "name",
    "stock_qty": "stock",
    "category_name": "category"
```

```

    }, inplace=True)

online.rename(columns={
    "sku": "code",
    "product": "name",
    "selling_price": "price",
    "product_category": "category"
}, inplace=True)

# -----
# 3. Clean product names
# -----

def clean_product_name(name):

    if pd.isna(name):
        return None

    name = str(name).strip().lower()

    name = re.sub(
        r"^[a-zA-Z0-9\s]",
        "",
        name
    )

    name = re.sub(
        r"\s+",
        " ",
        name
    )

    return name.title()

for df in [supplier, warehouse, online]:

    if "name" in df.columns:
        df["name"] = df["name"].apply(
            clean_product_name
        )

# -----
# 4. Validate product codes
# -----

# Example valid format: PROD-1001

code_pattern = r"^PROD-\d{4}$"

def validate_code(code):

```

```

    if pd.isna(code):
        return False

    return bool(
        re.match(
            code_pattern,
            str(code).strip().upper()
        )
    )

for df in [supplier, warehouse, online]:

    if "code" in df.columns:

        df["code"] = (
            df["code"]
            .astype(str)
            .str.strip()
            .str.upper()
        )

        df["valid_code"] = (
            df["code"].apply(validate_code)
        )

# -----
# 5. Remove invalid product codes
# -----

supplier = supplier[
    supplier["valid_code"] == True
]

warehouse = warehouse[
    warehouse["valid_code"] == True
]

online = online[
    online["valid_code"] == True
]

# -----
# 6. Duplicate detection
# -----

supplier = supplier.drop_duplicates(
    subset=["code"]
)

```

```

warehouse = warehouse.drop_duplicates(
    subset=["code"]
)

online = online.drop_duplicates(
    subset=["code"]
)

# -----
# 7. Standardize categories
# -----

category_map = {

    "mobiles": "Electronics",
    "mobile": "Electronics",
    "electronics": "Electronics",

    "laptop": "Electronics",
    "laptops": "Electronics",

    "clothes": "Fashion",
    "cloth": "Fashion",
    "fashion": "Fashion",

    "shoes": "Footwear",
    "footwear": "Footwear",

    "home appliance": "Home Appliances",
    "home appliances": "Home Appliances"
}

def standardize_category(category):

    if pd.isna(category):
        return "Unknown"

    category = str(category).strip().lower()

    return category_map.get(
        category,
        "Other"
    )

for df in [supplier, warehouse, online]:

    if "category" in df.columns:
        df["category"] = (
            df["category"]
            .apply(standardize_category)

```

```

    )

# -----
# 8. Convert price to numeric
# -----

for df in [supplier, online]:

    if "price" in df.columns:

        df["price"] = (
            df["price"]
            .astype(str)
            .str.replace("₹", "", regex=False)
            .str.replace(",", "", regex=False)
        )

        df["price"] = pd.to_numeric(
            df["price"],
            errors="coerce"
        )

# -----
# 9. Convert stock to numeric
# -----

if "stock" in warehouse.columns:

    warehouse["stock"] = pd.to_numeric(
        warehouse["stock"],
        errors="coerce"
    )

    warehouse["stock"] = (
        warehouse["stock"]
        .fillna(0)
        .clip(lower=0)
    )

# -----
# 10. Fuzzy matching
# -----

def fuzzy_similarity(a, b):

    return SequenceMatcher(
        None,
        str(a).lower(),
        str(b).lower()
    ).ratio()

```

```

print("\nFuzzy Matching Results:")

for _, s in supplier.iterrows():

    for _, o in online.iterrows():

        score = fuzzy_similarity(
            s["name"],
            o["name"]
        )

        if score >= 0.85:

            print(
                s["name"],
                "<->",
                o["name"],
                "Similarity:",
                round(score, 2)
            )

# -----
# 11. Price outlier detection
# -----

all_prices = pd.concat(
    [
        supplier[["code", "price"]],
        online[["code", "price"]]
    ],
    ignore_index=True
)

Q1 = all_prices["price"].quantile(0.25)
Q3 = all_prices["price"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

all_prices["price_outlier"] = (
    (all_prices["price"] < lower) |
    (all_prices["price"] > upper)
)

# Replace outliers with median

median_price = all_prices["price"].median()

```

```

all_prices.loc[
    all_prices["price_outlier"],
    "price"
] = median_price

# -----
# 12. Numerical normalization
# -----

min_price = all_prices["price"].min()
max_price = all_prices["price"].max()

if max_price != min_price:

    all_prices["normalized_price"] = (
        (all_prices["price"] - min_price)
        / (max_price - min_price)
    )

else:

    all_prices["normalized_price"] = 0

# -----
# 13. Create product master
# -----

product_master = pd.merge(
    supplier,
    warehouse[
        ["code", "stock"]
    ],
    on="code",
    how="outer",
    suffixes=("_supplier", "_warehouse")
)

product_master = pd.merge(
    product_master,
    online[
        ["code", "name", "price", "category"]
    ],
    on="code",
    how="outer",
    suffixes=("", "_online")
)

# -----
# 14. Fill missing values

```

```

# -----

product_master["stock"] = (
    product_master["stock"]
    .fillna(0)
)

product_master["category"] = (
    product_master["category"]
    .fillna(
        product_master["category_online"]
    )
)

product_master["category"] = (
    product_master["category"]
    .fillna("Unknown")
)

# Select useful columns

final_product_master = product_master[
    [
        "code",
        "name",
        "price",
        "category",
        "stock"
    ]
]

# -----
# 15. Remove duplicates
# -----

final_product_master = (
    final_product_master
    .drop_duplicates(subset=["code"])
)

# -----
# 16. Save CSV
# -----

final_product_master.to_csv(
    "product_master.csv",
    index=False
)

# -----

```

17. Save JSON

```
final_product_master.to_json(  
    "product_master.json",  
    orient="records",  
    indent=4  
)  
  
print("\nProduct integration completed!")  
print(final_product_master)
```

```
print("\nFiles generated:")  
print("1. product_master.csv")  
print("2. product_master.json")
```

Example Input

Supplier

Product Code	Product Name	Unit Price	Category
PROD-1001	Samsung Galaxy S25	₹75000	mobiles
PROD-1002	Dell Laptop	₹60000	laptop
PROD-1003	Nike Shoes	₹5000	shoes

Warehouse

Prod Code	Prod Name	Stock Qty	Category
PROD-1001	Samsung Galaxy S25	20	Electronics
PROD-1002	Dell Laptp	15	Laptops
PROD-1003	Nike Shoe	30	Footwear

Online Store

SKU	Product	Selling Price	Product Category
PROD-1001	Samsung Galaxy S25	₹76000	Mobile
PROD-1002	Dell Laptop	₹62000	Electronics
PROD-1003	Nike Shoes	₹5100	Shoes

```

Duplicate Products:
Supplier: 0
Warehouse: 0
Online Store: 0

=====
FUZZY MATCHING RESULTS
=====
Samsung Galaxy S25 <-> Samsung Galaxy S25 Similarity: 1.0
Dell Laptop <-> Dell Laptop Similarity: 1.0
Nike Shoes <-> Nike Shoes Similarity: 1.0

=====
PRICE OUTLIER DETECTION
=====
Empty DataFrame
Columns: [code, price, price_outlier]
Index: []

=====
NORMALIZED PRICE
=====
   code  price  normalized_price
0  PROD-1001  75000         0.985915
1  PROD-1002  60000         0.774648
2  PROD-1003   5000         0.000000
3  PROD-1001  76000         1.000000
4  PROD-1002  62000         0.802817
5  PROD-1003   5100         0.001408

=====
FINAL PRODUCT MASTER
=====
   code          name  price  category  stock
PROD-1001  Samsung Galaxy S25  75000  Electronics    20
PROD-1002      Dell Laptop  60000  Electronics    15
PROD-1003      Nike Shoes   5000   Footwear    30

=====
INTEGRATION COMPLETED
=====
Files generated:
1. product_master.csv
2. product_master.json
PS C:\Users\sambh\Videos\product_integration>

```